

CO1 - TECHNICAL PRESENTATION

Database Management Systems

Course Code: CSA0504

Name : SANTHOSH E

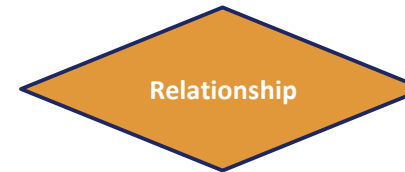
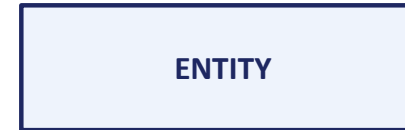
Reg No : 192511470

QUESTION 1 · ER MODELING

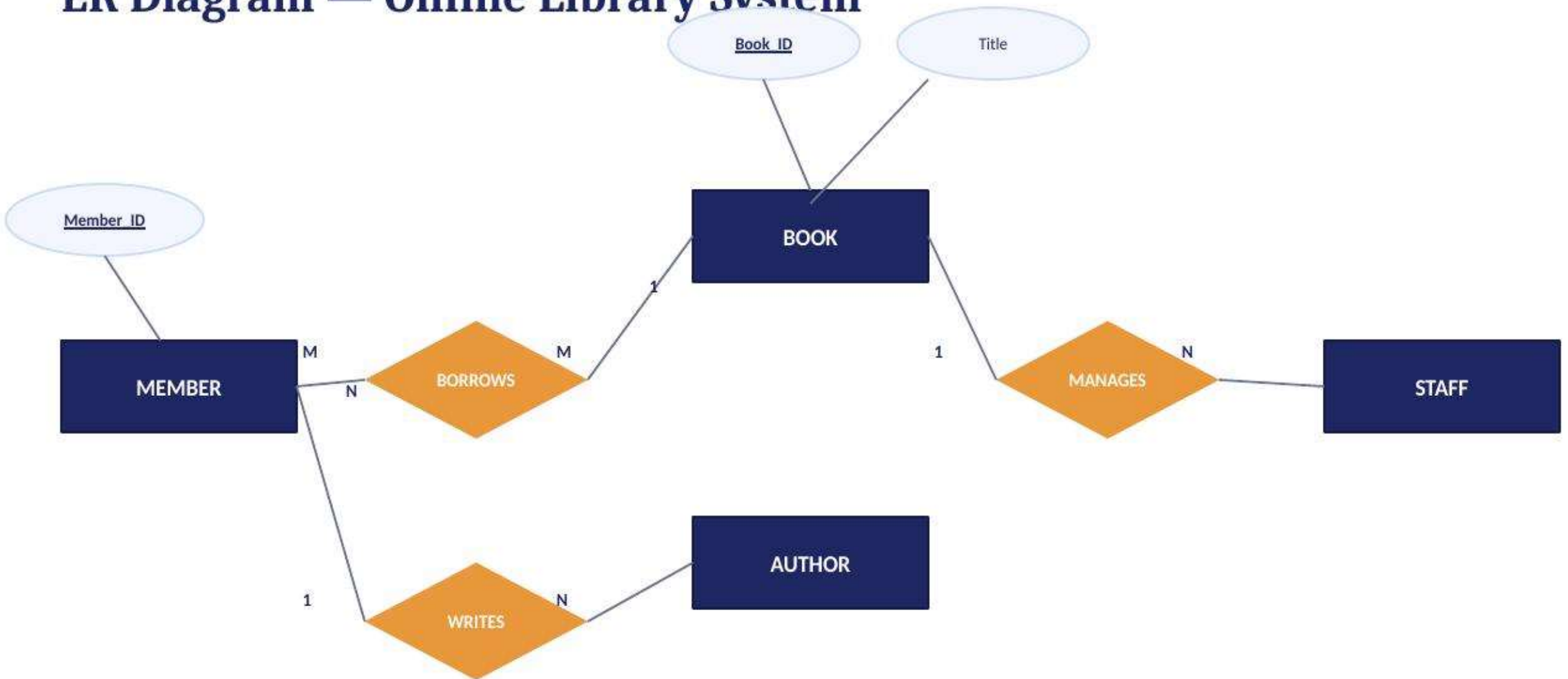
Entity-Relationship Diagrams: Core Concepts

- 1 Entity — a real-world object with independent existence, drawn as a rectangle (e.g., BOOK, MEMBER)
- 2 Attribute — a property that describes an entity, drawn as an oval (e.g., Title, Email)
- 3 Relationship — an association between two or more entities, drawn as a diamond (e.g., BORROWS)
- 4 Key Attribute — uniquely identifies each entity instance, shown underlined (e.g., Book_ID)
- 5 Cardinality — defines participation ratio between entities: 1:1, 1:N, or M:N

ER Notation Legend



ER Diagram — Online Library System



Entities: Member, Book, Author, Staff | Relationships: Borrows (M:N), Writes (1:N), Manages (1:N)

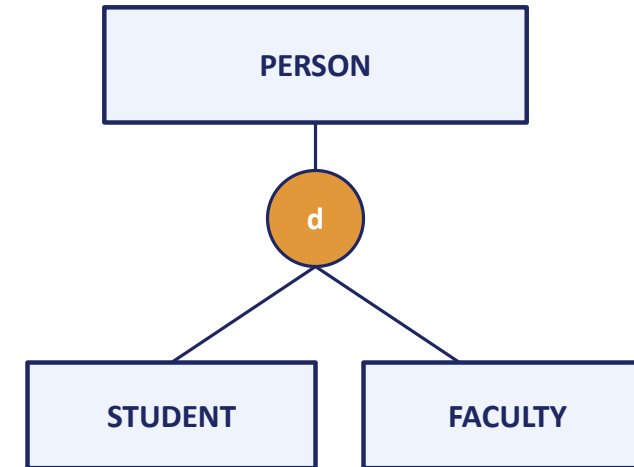
Explaining the Library System Components

- 1 BOOK: key attribute Book_ID, with Title, ISBN, Genre and Copies_Available describing each title
- 2 MEMBER: key attribute Member_ID, with Name, Email and Membership_Type
- 3 BORROWS: an identifying M:N relationship linking Member and Book, carrying Issue_Date, Due_Date and Return_Date
- 4 AUTHOR (M:N via WRITES) and PUBLISHER (1:N via PUBLISHES) enrich Book with authorship and source details
- 5 LIBRARIAN manages Book records through the MANAGES relationship, ensuring accurate inventory tracking

QUESTION 2 · EER MODEL

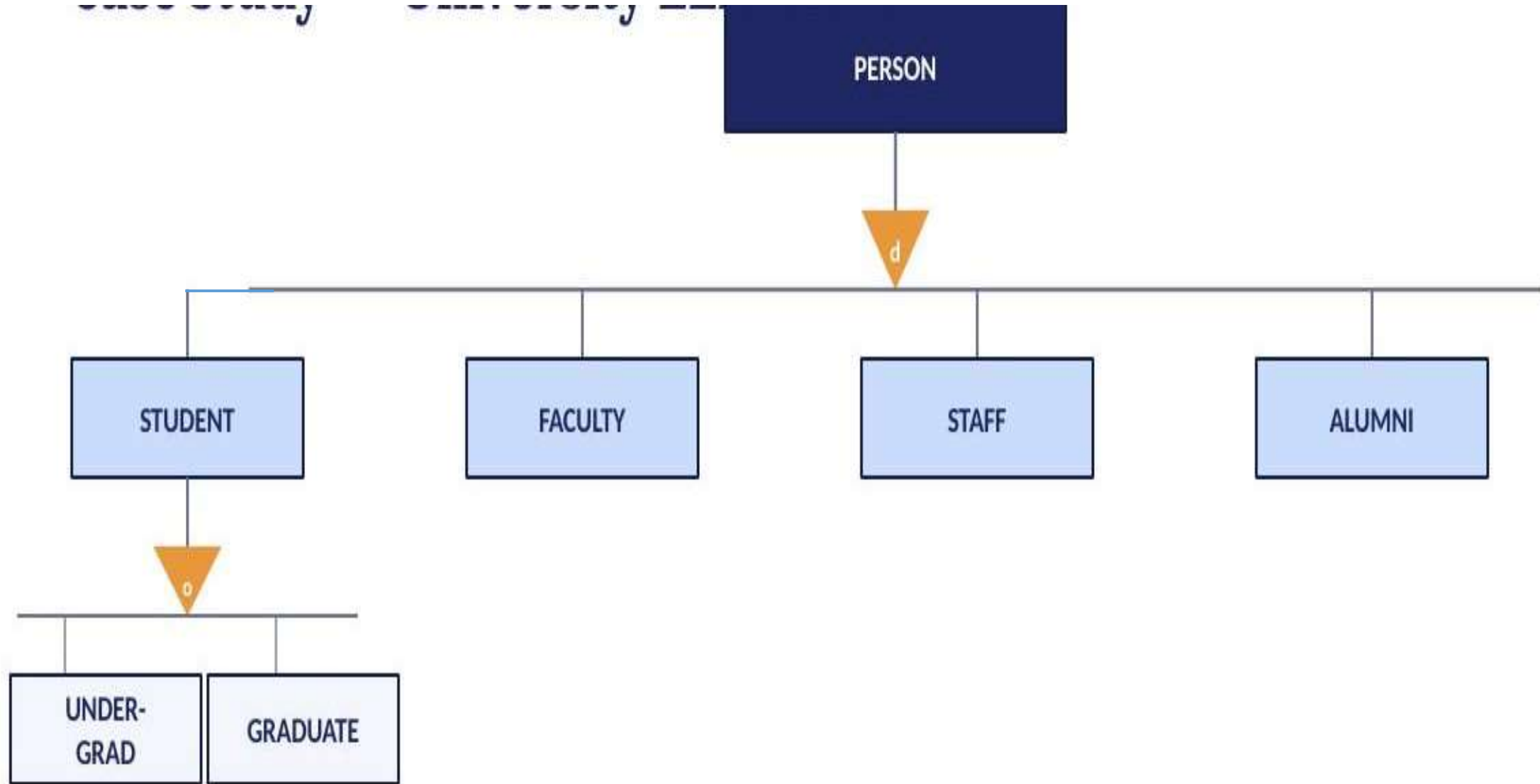
Generalization & Specialization

- 1 Generalization: a bottom-up process that combines lower-level entities sharing common features into one higher-level superclass
- 2 Specialization: a top-down process that divides a superclass into subclasses based on distinguishing characteristics
- 3 Attribute Inheritance: every subclass inherits all attributes and relationships of its superclass
- 4 Disjoint (d) vs Overlapping (o): an instance belongs to exactly one subclass (d), or possibly more than one (o)
- 5 Total vs Partial Participation: total requires every superclass instance to belong to a subclass; partial does not



*Subclass hierarchy
(University case study →)*

EER Diagram — University Case Study

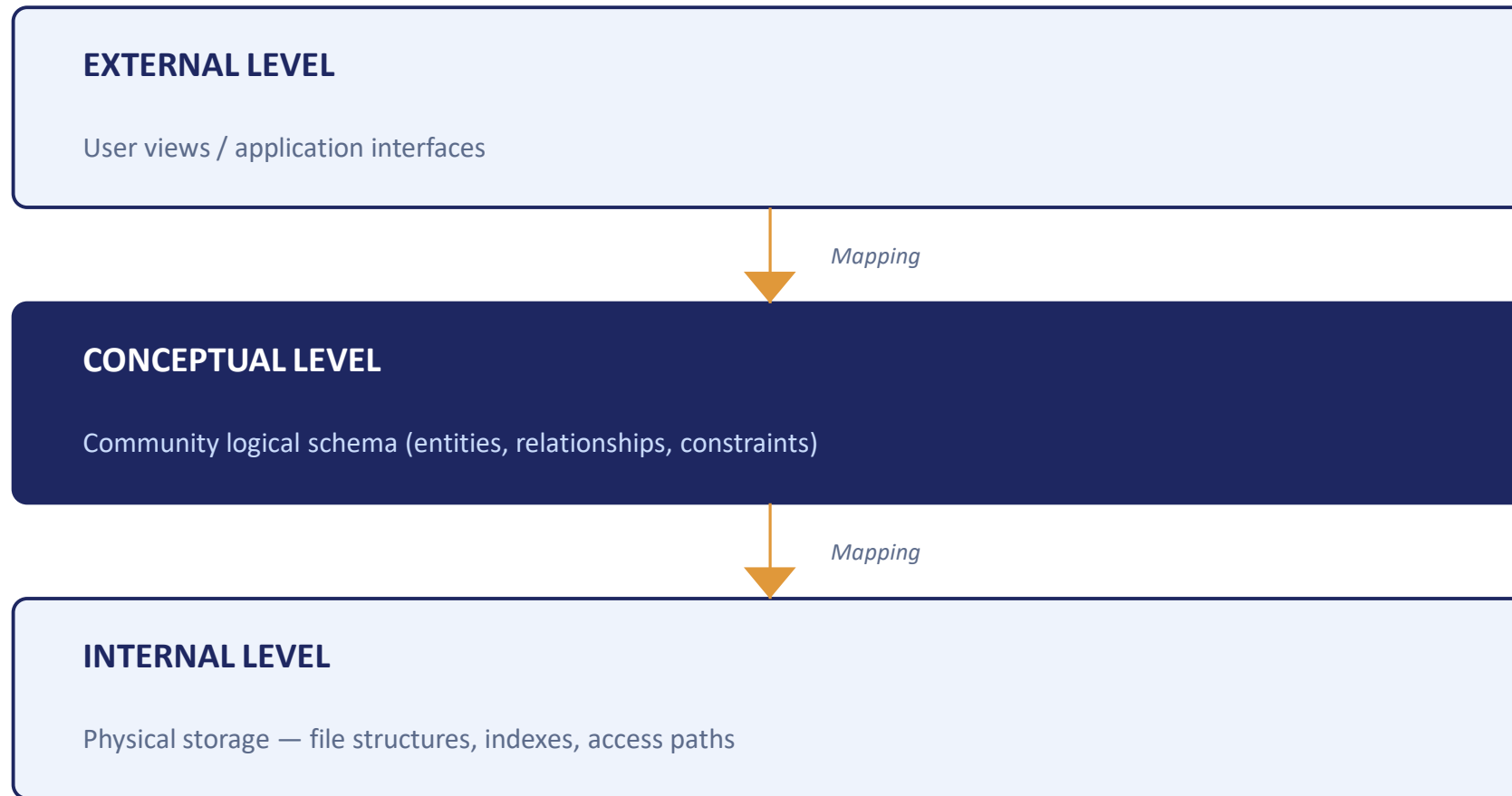


Interpreting the University EER Diagram

- 1 PERSON is the superclass holding attributes common to every member of the university community
- 2 Specialization into STUDENT, FACULTY and STAFF uses a disjoint (d) constraint — each person belongs to exactly one subclass
- 3 Total participation ensures every PERSON instance is classified into one of the three subclasses; none remain unclassified
- 4 STUDENT is further specialized into UNDERGRADUATE and POSTGRADUATE with an overlapping (o) constraint for dual-enrolled students
- 5 Each subclass inherits Person_ID, Name and Address from PERSON while adding its own specific attributes

QUESTION 3 · DATA INDEPENDENCE

The Three-Schema Architecture



Logical vs. Physical Data Independence

Logical Data Independence

- Capacity to change the conceptual schema without altering external schemas or application programs
- Protects applications from changes like adding a new attribute or entity
- Harder to achieve — application logic is closely tied to logical structure
- Example: adding a Membership_Type column to MEMBER without breaking existing programs

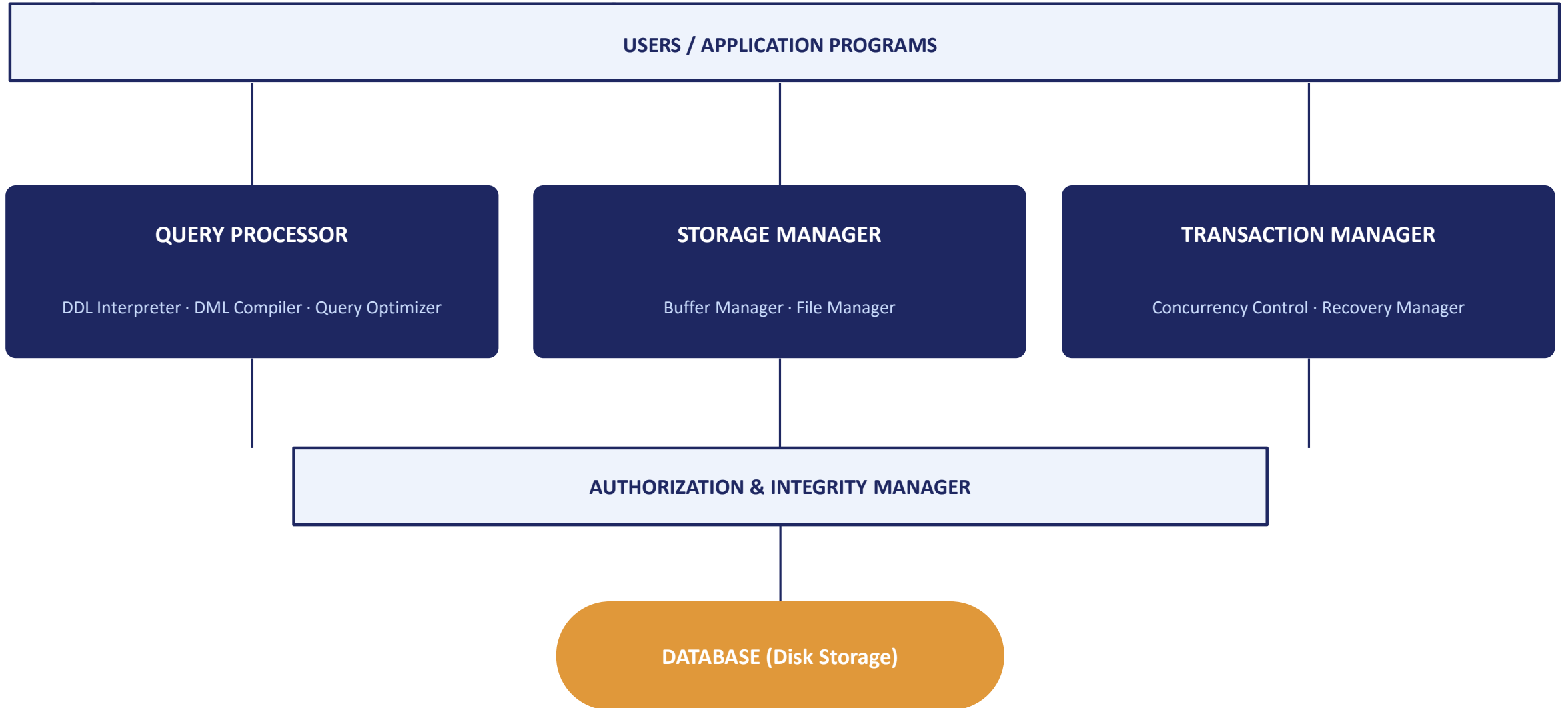
Physical Data Independence

- Capacity to change the internal (storage) schema without altering the conceptual schema
- Protects the conceptual level from changes like new indexes or file reorganization
- Easier to achieve — storage details are hidden beneath the conceptual layer
- Example: switching Book records from sequential to indexed file storage transparently

Significance in DBMS Architecture

- 1 Shields application programs from underlying storage and structural changes, cutting maintenance effort and cost
- 2 Lets database administrators tune storage or indexing for performance without disrupting end users
- 3 Supports smooth schema evolution as organizational and business requirements grow over time
- 4 Improves portability, scalability, and long-term reliability of database-driven systems
- 5 Forms the theoretical foundation of the ANSI/SPARC three-schema architecture used across modern DBMS

QUESTION 4 · DBMS ARCHITECTURE



Query Processor & Storage Manager

- 1 Query Processor interprets DML statements from users and converts them into low-level execution instructions
- 2 Its DDL Interpreter records schema definitions, while the DML Compiler translates queries into an evaluation plan
- 3 The Query Optimizer selects the most efficient execution strategy among equivalent plans
- 4 Storage Manager acts as the interface between application programs and data stored on disk
- 5 It comprises a Buffer Manager (manages in-memory data movement) and a File Manager (manages disk space allocation and file structures)

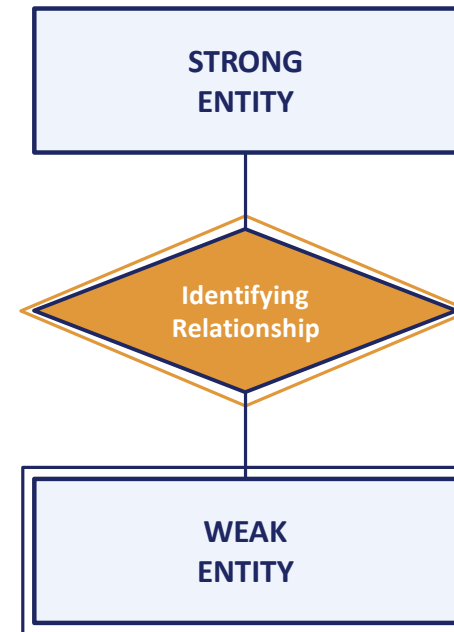
Transaction Manager & Concurrency Control

- 1 Transaction Manager ensures every transaction satisfies the ACID properties: Atomicity, Consistency, Isolation, Durability
- 2 It coordinates concurrent execution of multiple transactions so results remain correct and conflict-free
- 3 The Concurrency Control component applies locking or timestamp-based protocols to prevent anomalies
- 4 The Recovery Manager restores the database to a consistent state after a crash, using transaction logs
- 5 Transaction Manager works closely with the Storage Manager to safely commit or roll back changes

QUESTION 5 • STRONG VS WEAK ENTITIES

Understanding Strong & Weak Entities

- 1 Strong Entity: has its own primary key and can exist independently (e.g., EMPLOYEE, ORDER)
- 2 Weak Entity: has no primary key of its own; its existence depends on a related strong (owner) entity
- 3 Partial Key (Discriminator): distinguishes weak-entity instances that share the same owner, shown with a dashed underline
- 4 Identifying Relationship: connects a weak entity to its owner; drawn as a double-outlined diamond
- 5 Notation: weak entity = double-outlined rectangle; identifying relationship = double-outlined diamond



Strong vs. Weak Entities — Comparison

Aspect	Strong Entity	Weak Entity
Primary key	Has its own primary key	No primary key; uses a partial key
Existence	Exists independently	Existence depends on owner entity
Notation	Single-outlined rectangle	Double-outlined rectangle
Relationship	Regular relationship (diamond)	Identifying relationship (double diamond)
Example	EMPLOYEE, ORDER	DEPENDENT, ORDER_ITEM

Real-World Examples



1

ORDER_ITEM cannot exist without its parent ORDER — deleting the order removes its line items too



2

DEPENDENT records (e.g., a child or spouse) only make sense in the context of an EMPLOYEE

THANK YOU